vec_dot_q4_K_q8_K 多平台向量实现对比与RVV扩展指令建议

概述

分析目标: `ggml_vec_dot_q4_K_q8_K` — Q4_K 权重与 Q8_K 激活值的向量点积，llama.cpp 推理中占计算量 ~16%（Qwen2.5-0.5B Q4_K_M 模型）。

基准实现: RVV VLEN=512 (SEW=8, LMUL=1, 64 elements/register)

分析平台: x86 AVX/AVX2 (256-bit), ARM NEON/SVE (128-512-bit), LoongArch LASX/LSX (128/256-bit), Power VSX/MMA (128-bit + 512-bit ACC), S390X Z-Vector (128-bit), WASM SIMD (128-bit)

BBV数据: 未提供，收益为BB内理论估算

归一化基准: 所有比较归一化到 RVV VLEN=512bit, SEW=8bit。RVV 每 j 迭代处理 64 元素（32B Q4 + 64B Q8），需 14 条指令。

指令方案汇总

优先级	扩展指令	来源平台	**BB**内收益	实现难度	**RVV**现状
P0	循环重构使用 `vwmacc_vx` (EEW=16→32)	x86 AVX2 `PMADDWD` 模式	BB内减少 35.7%（主循环BB）	低（纯代码重构）	规范已有，代码未利用

优先级	扩展指令	来源平台	**BB**内收益	实现难度	**RVV**现状
P0	融合点积 `vwmaccbb_vv` (u8×i8→i16, pairwise)	x86 `PMADDUBSW`，ARM `vdotq_s32`，Power `vec_msum`，S390X `VMSL`，LoongArch `madd_h_b`	BB内减少14.3%（主循环BB）	中	无此融合操作
P1	融合点积+缩放 `vdot_scale` (i8×i8→i32, 含pairwise + scale)	ARM `vdotq_s32`，Power `vec_msum`，x86 `PMADDUBSW` + `PMADDWD` 链	BB内减少50%（主循环BB）	高	需vwmul→i16 + vredsum→scalar + scalar mul
P2	矩阵乘累加 (MMA)	ARM `svmmla_s32`，Power `xvf32gerpp`	BB内减少93%（理论值）	极高	无矩阵级操作，需专用累加器
P2	专用累加器寄存器	Power MMA ACC, LoongArch 向量域累积	消除reduce-to-scalar	高	无专用累加器

收益计算方式（无 BBV 数据）： - 上表中的"BB内收益"仅反映单个主循环 BB 范围内的指令减少比例 - 计算公式：`（原指令数 - 新指令数）/ 原指令数 × 100%` - 主循环 BB = 每个 j 迭代的循环体（14 条指令/64 元素） - 建议通过 `./tools/profile_to_dfg.sh` 获取 BBV 数据后重新估算整体收益

关键发现: P0 循环重构（纯软件优化，无需新硬件）即可减少 35.7% 指令。若叠加 P0 融合点积扩展，总减少达 50%。P2 级扩展为长期架构演进方向。

基准RVV实现分析

算法概述

`vec_dot_q4_K_q8_K` 计算 Q4_K 量化权重与 Q8_K 量化激活值的点积。每个 super-block (QK_K=256 elements) 包含 4 个 subblock（各 64 elements），每个 subblock 有独立的 scale factor。

循环结构（**per j iteration, 64 elements**）

```
// --- Lower nibbles × Q8[0..31] ---
vuint8m1_t q4_x  = __riscv_vle8_v_u8m1(q4, 32);              // 1: Load 32B Q4
vint8m1_t  q8_lo = __riscv_vle8_v_i8m1(q8, 32);               // 2: Load 32B Q8
vint8m1_t  q4_lo = __riscv_vreinterpret_v_u8m1_i8m1(           // 3: Extract low nibble
    __riscv_vand_vx_u8m1(q4_x, 0x0F, 32));
vint16m2_t qv_lo = __riscv_vwmul_vv_i16m2(q4_lo, q8_lo, 32);  // 4: i8×i8→i16 widening mul
vint16m1_t vs_lo = __riscv_vredsum_vs_i16m2_i16m1(             // 5: Reduce 32i16→1i16
    qv_lo, vzero, 32);
sum_1 += __riscv_vmv_x_s_i16m1_i16(vs_lo) * scales[2*j + 0]; // 6: Scalar scale mul

// --- Upper nibbles × Q8[32..63] ---
vint8m1_t  q8_hi = __riscv_vle8_v_i8m1(q8 + 32, 32);         // 7: Load 32B Q8
vint8m1_t  q4_hi = __riscv_vreinterpret_v_u8m1_i8m1(           // 8: Extract high nibble
    __riscv_vsrl_vx_u8m1(q4_x, 0x04, 32));
vint16m2_t qv_hi = __riscv_vwmul_vv_i16m2(q4_hi, q8_hi, 32);  // 9: i8×i8→i16 widening mul
vint16m1_t vs_hi = __riscv_vredsum_vs_i16m2_i16m1(             // 10: Reduce 32i16→1i16
    qv_hi, vzero, 32);
sum_2 += __riscv_vmv_x_s_i16m1_i16(vs_hi) * scales[2*j + 1]; // 11: Scalar scale mul
// Pointer updates: q4 += 32; q8 += 64;                        // 12-14
```

关键瓶颈

系统性问题**:** 过早 **reduce-to-scalar**

当前实现每个 nibble half 都做 `vredsum` 归约到标量，然后做标量 scale 乘法。这意味着每个 j 迭代做 2 次 vredsum（共 8 次/super-block），将 32 个有用的 i16 值压缩成 1 个标量，丢失了所有向量并行性。

x86 AVX2 和 ARM NEON 都保持中间结果在向量域（i32 精度），仅在 super-block 结束时做一次 float 转换。RVV 在每个 subblock half 就 reduce 到标量，无法利用向量执行单元的吞吐。

寄存器与精度分析

参数	值
VLEN	512 bit
SEW	8 bit
LMUL	1
每寄存器元素数	64 × int8
vwmul 输出	32 × int16 (LMUL=2)
vredsum 输出	1 × int16 (LMUL=1)
中间精度	**int16**（最大值 ±6144，范围 ±32767）

精度差距: 多平台（ARM, Power, S390X, x86 AVX2）的融合指令直接产生 int32 精度结果，避免 int16 中间精度的潜在溢出。32 次 i8×i8 累加最大值 ±6144，i16 范围 ±32767，虽未溢出但安全裕度仅 5.3×。

各平台对比分析

1. x86 AVX/AVX2

核心特点: 256-bit YMM 寄存器（AVX2），`PMADDUBSW` / `PMADDWD` 两条融合指令构成 i8→i16→i32 的完整链路。

高价值指令:

指令	功能	**RVV**现状
`_mm256_maddubs_epi16` (PMADDUBSW)	u8×i8→i16 pairwise dot	vand + vwmul（2 inst，无 pairwise）
`_mm256_madd_epi16` (PMADDWD)	i16×i16→i32 pairwise MAC	vredsum + scalar mul（2 inst，reduce 到标量）
`_mm256_fmadd_ps` (VFMADD132PS)	float FMA	标量 mul + add（2 ops）

关键洞察 **— AVX2** 保持向量域累积:

AVX2 的 `PMADDUBSW` → `PMADDWD` 链将 u8→i16→i32 完全保持在向量域，最终产生 8 个 i32 累积到 `sumi` 寄存器。跨 4 个 subblock 累加仅需 4 次 `add_epi32`，在 super-block 结束时做一次 `cvtepi32_ps` + `fmadd`。

Per-iteration 对比（**64 elements**）：

步骤	**AVX2**	**AVX2 count**	**RVV**	**RVV count**	差距
Scale broadcast	shuffle × 2	2	(embedded)	0	AVX2 +2
Load Q4 (32B)	loadu_si256	1	vle8_v_u8m1	1	same
Nibble extract	and + srli+and	3	vand + vsrl	2	RVV -1
Load Q8 (64B)	loadu_si256 × 2	2	vle8_v_i8m1 × 2	2	same
u8×i8 → i16 dot	**maddubs_epi16 × 2**	**2**	**vand + vwmul × 2**	**4**	**RVV +2**
i16×i16 scale → i32	**madd_epi16 × 2**	**2**	**vredsum + scalar mul × 2**	**4**	**RVV +2**
Accumulate	add_epi32 × 2	2	(scalar +=)	0	

步骤	**AVX2**	**AVX2 count**	**RVV**	**RVV count**	差距
					AVX2 +2
Total		**14**		**14**	**same count**

关键差异: 指令数相同（14 vs 14），但 质量差距巨大： - AVX2 保持 8×i32 向量域累积，每 super-block 仅 1 次 float 转换 + 1 次 FMA - RVV 做 8 次 vredsum/super-block（纯浪费 vector 带宽），标量累加无法利用向量执行单元

建议扩展: `vwmaccbb_vv` — 融合零扩展 + 乘法 + 成对加法（对应 PMADDUBSW）。

---

2. ARM NEON/SVE

核心特点: NEON 128-bit Q 寄存器；SVE 可缩放（128-2048 bit）。`vdotq_s32` 直接产生 int32 精度点积。

高价值指令:

指令	功能	**NEON/SVE**	**RVV**现状
`vdotq_s32` / `svdot_lane_s32`	i8×i8→i32 dot + pairwise + accumulate	NEON/SVE2	vwmul→i16 + vredsum→scalar（2 inst，i16精度）
`svmmla_s32`	4×4 i8×i8→i32 matrix MAC	SVE2+MATMUL	无矩阵级操作
`svmla_lane_s32_x`	Lane-indexed i32 MAC	SVE2	vwmacc_vx（已有但未利用）

关键洞察 **— ARM** 直接产生 **int32**:

```
// ARM NEON: 1 条指令，直接产生 4×int32（pairwise sum of 4 products each）
dotl = vdotq_s32(vdupq_n_s32(0), q4l, q8l);
// 无需 int16 中间步骤

// RVV: 2 条指令，产生 1×int16（丢失精度和并行性）
qv_lo = vwmul_vv_i16m2(q4_lo, q8_lo, 32);    // 32×i16
vs_lo = vredsum_vs_i16m2_i16m1(qv_lo, vzero, 32); // → 1×i16
```

Per-iteration 对比（归一化到 **64 elements**）：

平台	迭代次数	总指令数	中间精度	累积域
RVV	1× (512-bit)	14	i16	标量
NEON	4× (128-bit)	40	**i32**	标量
SVE VL=256	2× (256-bit)	24	**i32**	标量

NEON/SVE 总指令数更多（受限于寄存器宽度），但 **int32** 精度 保证了数值安全，且 `vdotq_s32` 的 pairwise sum 减少了后续归约负担。

建议扩展: `vdot_u8_i8_i32` — 融合 u8×i8→i32 点积 + pairwise sum（对应 vdotq_s32）。

---

3. LoongArch LASX/LSX

核心特点: LASX 256-bit XR 寄存器；LSX 128-bit VR 寄存器。`madd_h_b` / `madd_w_h` 构成 i8→i16→i32 的 MAC 链。

高价值指令:

指令	功能	**RVV**现状
`lasx_madd_h_b`	i8×i8→i16 MAC（向量化）	vwmul + vredsum（2 inst，reduce到标量）
`lasx_madd_w_h`	i16×i16→i32 MAC（向量化）	无等价（已在标量域）
`lasx_xvldrepl.w`		scalar load（嵌入在 mul 中）

指令	功能	**RVV**现状
	加载 4B 并复制到 32B 全通道	

关键洞察 **— LASX** 向量域累积:

LASX 通过 `madd_h_b` 在向量域内完成 i8×i8→i16 MAC（3 操作数: `xd.H[i] += xs1.B[i] * xs2.B[i]` ，无 scale 参数），累加器始终保持向量寄存器状态，避免了 RVV 的 reduce-to-scalar 问题。Scale 广播和乘法需额外指令。

```
// LASX: 向量域内完成 multiply + accumulate
xacc_lo = lasx_madd_h_b(xacc_lo, q4_lo, q8_lo);  // 1 条 MAC（向量域, i8→i16）
scale_lo = lasx_xvldrepl.w(scales[2j]);           // 广播 scale
xacc_lo = lasx_xvmul_w(xacc_lo, scale_lo);        // 向量 scale 乘法
// xacc_lo 保持 i16 向量状态, 跨迭代累加
```

归一化指令对比（**64 elements**）:

平台	指令数	向量域指令	标量指令	相对 **RVV**
RVV (VLEN=512)	14	7	7	1.00×
LASX (256-bit)	~11	10	1	**0.79×**

LASX 的优势来自向量域累积（避免 reduce-to-scalar），但需额外 scale 乘法指令。修正后 LASX 比 RVV 少约 21.4%。

---

4. Power VSX (POWER10)

核心特点: VSX 128-bit VSR 寄存器；POWER10 MMA 提供 512-bit 专用累加器，4×4 子矩阵操作。

高价值指令:

指令	功能	**RVV**现状
`vec_msum`	u8×i8→i32 乘加 + pairwise sum	vwmul + vredsum + scalar mul（3 inst）

指令	功能	**RVV**现状
`xvf32gerpp`	4×4 i8×i8→i32 MAC（MMA）	无矩阵级操作
MMA ACC	512-bit 专用累加器（无需显式 reduce）	通用向量寄存器

关键洞察 **— Power** 原生 **i32** 精度:

`vec_msum` 直接将 i8×i8→i32，并做 4 元素 pairwise sum + accumulate，单指令完成。MMA 进一步提供 4×4 矩阵级 MAC。

```
# Power: 单指令 i8→i32
vec_msum  v6, v2, v4, v6   # v6[i] += sum(v2[4i..4i+3] * v4[4i..4i+3]) → 4×i32
```

对比 RVV 需要 3 条指令（vwmul + vredsum + scalar mul）且仅得到 i16 精度。

建议扩展: `vbmulacc` — 字节乘加融合（对应 vec_msum），i8×i8→i32 直接产出。

---

5. S390X Z-Vector

核心特点: 128-bit VR 寄存器（VEF1/2/3）。`VMSL` / `VMSLG` 提供乘加累加。

高价值指令:

指令	功能	**RVV**现状
`VMSL`	乘法 + 求和（i8→i16 或 i16→i32，含累加）	vwmul + vredsum（2 inst）
`VMSLG`	扩展乘加累加（累加到 i64，防溢出）	无等价
`VSUM` / `VSUMQ`	跨通道归约（直接到标量）	vredsum + vmv_x_s（2 inst）

关键洞察 **—** 向量域累加 **+** 宽精度:

S390X 的 `VMSL` 可以在向量域内完成乘加累加，仅在需要最终结果时做一次 `VSUM` 归约。`VMSLG` 将累加宽度扩展到 i64，消除所有溢出风险。

---

6. WASM SIMD

核心特点: 128-bit V128 寄存器。`i32x4.dot_i16x8_s` 提供成对点积（i16×i16→i32）。

平台定位: WASM SIMD 是 RVV 的子集。RVV 在寄存器宽度、归约指令、宽化乘法等方面全面领先。WASM SIMD 的 `i32x4.dot_i16x8_s` 与其他平台的融合点积指令概念一致，但缺少 RVV 的可变长度向量、全宽度归约等能力。

无显著 **RVV** 缺失指令: WASM SIMD 的所有操作在 RVV 中均有等价或更优的实现。

跨平台共识分析

共识 1: 融合点积指令（i8×i8→直接精度 + accumulate）

来源: ARM `vdotq_s32`, x86 `PMADDUBSW`, Power `vec_msum`, LoongArch `madd_h_b`, S390X `VMSL`

所有 5 个非-WASM 平台都有某种形式的融合点积指令，将 u8/i8 乘法 + 累积（部分或全部）融合为 1 条指令。RVV 需要 2 条（`vwmul` + `vredsum`）。

平台	指令	输出精度	**Pairwise**	**Accumulate**
x86	PMADDUBSW	i16	Yes	No
ARM	vdotq_s32	**i32**	Yes	Yes
Power	vec_msum	**i32**	Yes	Yes
LoongArch	madd_h_b	i16	No	Yes
S390X	VMSL	i16	No	Yes

平台	指令	输出精度	**Pairwise**	**Accumulate**
RVV	**vwmul + vredsum**	**i16**	**No**	**No (标量)**

共识 2: 向量域累积（减少 reduce-to-scalar）

来源: x86 AVX2, ARM SVE, LoongArch LASX, Power MMA, S390X

所有平台都倾向于在向量域内维护累加器，仅在最终阶段做标量归约。RVV 当前实现在每个 subblock half 就 reduce 到标量。

共识 3: 直接 i32 精度（避免 i16 中间步骤）

来源: ARM `vdotq_s32` → i32, Power `vec_msum` → i32, S390X `VMSLG` → i64, x86 `PMADDWD` → i32

4/6 平台的融合指令直接产出 i32 或更宽精度。RVV 止步于 i16，安全裕度有限。

RVV扩展指令建议详细说明

[P0] 循环重构使用 `vwmacc_vx` (EEW=16→32) — 纯软件优化

核心发现: RVV 规范已有 `vwmacc_vx vd, rs1, vs2`（`vd.W[i] += sext(rs1) * vs2.H[i]`），可将 i16→i32 的 scale multiply 保持在向量域。当前代码未利用此指令。

重构前（**14 inst/j iteration**）：

```
// Low nibble path
q4_x  = vle8_v_u8m1(q4, 32);               // 1
q8_lo = vle8_v_i8m1(q8, 32);               // 2
q4_lo = vand_vx_u8m1(q4_x, 0x0F, 32);      // 3
qv_lo = vwmul_vv_i16m2(q4_lo, q8_lo, 32);  // 4: → 32×i16
vs_lo = vredsum_vs_i16m2_i16m1(qv_lo, vzero, 32); // 5: → 1×i16 (浪费)
sum_1 += vmv_x_s_i16m1_i16(vs_lo) * scales[2*j]; // 6: scalar

// High nibble path (7 more instructions)
```

```
q8_hi = vle8_v_i8m1(q8+32, 32);            // 7
q4_hi = vsrl_vx_u8m1(q4_x, 4, 32);        // 8
qv_hi = vwmul_vv_i16m2(q4_hi, q8_hi, 32);  // 9
vs_hi = vredsum_vs_i16m2_i16m1(qv_hi, vzero, 32); // 10
sum_2 += vmv_x_s_i16m1_i16(vs_hi) * scales[2*j+1]; // 11
// Total: 11 instructions + 3 pointer ops = 14
```

重构后（**9 inst/j iteration**）：

```
vint32m4_t acc = __riscv_vmv_v_x_i32m4(0, 32); // 初始化 i32 累加器

for (int j = 0; j < QK_K/64; ++j) {
    // Load (3 inst)
    q4_x  = __riscv_vle8_v_u8m1(q4, 32);
    q8_lo = __riscv_vle8_v_i8m1(q8, 32);
    q8_hi = __riscv_vle8_v_i8m1(q8 + 32, 32);

    // Fused dot + scale accumulate (6 inst)
    q4_lo = __riscv_vand_vx_u8m1(q4_x, 0x0F, 32);
    tmp16 = __riscv_vwmul_vv_i16m2(q4_lo, q8_lo, 32);     // 32×i16
    __riscv_vwmacc_vx_i32m4(acc, scales[2*j], tmp16, 32); // i32 += i16×scale → vector MAC

    q4_hi = __riscv_vsrl_vx_u8m1(q4_x, 4, 32);
    tmp16 = __riscv_vwmul_vv_i16m2(q4_hi, q8_hi, 32);
    __riscv_vwmacc_vx_i32m4(acc, scales[2*j+1], tmp16, 32);

    q4 += 32; q8 += 64;
}
// Post-loop: reduce acc (i32 vector) to scalar once
```

指令减少: 14 → 9 inst, **BB**内减少 **35.7%**

额外收益: - i32 累加精度（消除 i16 溢出风险） - 向量域累积（仅 1 次最终 reduce/super-block） - 为后续扩展（vwmaccbb_vv）提供基础

约束: 需要 LMUL=4 的 vint32m4_t 累加器，占用 4 个向量寄存器组。VLEN=512 下可容纳 16×i32，足够覆盖 32 个 i16→i32 的转换。

[P0] `vwmaccbb_vv` — 融合零扩展 + 乘法 + 成对加法 (对应 `PMADDUBSW`)

指令定义:

```
格式: vwmaccbb_vv vd, vs2, vs1, vm
语义: For each element pair (2k, 2k+1):
      vd.W[2k] += zext(vs2.B[2k]) * sext(vs1.B[2k])
      vd.W[2k+1] += zext(vs2.B[2k+1]) * sext(vs1.B[2k+1])
      // (非 pairwise add, 保持独立元素；或可选 pairwise 模式)
输入: vd (i16 vector, LMUL=2), vs2 (u8 vector), vs1 (i8 vector)
输出: vd (i16 vector, 累加结果)
约束: vd.EEW=16, vs2/vs1.EEW=8, vd.LMUL = vs2.LMUL * 2
```

对应: Intel `PMADDUBSW` (x86 SSE4.1/AVX2), ARM `vdotq_s32` (部分语义), Power `vec_msum`, S390X `VMSL`

与 **P0** 循环重构组合: - 单独使用: 14 → 12 inst（省去 2 条 vand/vsrl × 2） - 与重构组合: 9 → 7 inst, **BB**内减少 **50%**

[P1] `vdot_scale` — 融合 i8×i8→i32 点积 + pairwise + scale (跨平台共识指令)

指令定义:

```
格式: vdot_scale vd, vs2, vs1, rs1, vm
语义: For each group of 4 elements:
      vd.W[k] += zext(vs2.B[4k..4k+3]) · sext(vs1.B[4k..4k+3]) * sext(rs1)
      (pairwise dot of 4 u8×i8 products, accumulated to i32, multiplied by scalar scale)
输入: vd (i32 vector, LMUL=1), vs2 (u8 vector), vs1 (i8 vector), rs1 (scalar i8 scale)
输出: vd (i32 vector)
约束: vd.EEW=32, vs2/vs1.EEW=8, VL_processed = VL / 4
```

对应: ARM `vdotq_s32` + scale multiply, Power `vec_msum` + `xvmulasp`, x86 `PMADDUBSW` + `PMADDWD` 链

应用场景: 将当前代码中 `vand + vwmul + vredsum + scalar mul` （4 条指令）替换为 1 条指令。

性能对比 (per nibble path, 32 elements):

```
Before (current RVV):
  q4_lo = vand_vx_u8m1(q4_x, 0x0F, 32)          // 1 inst
  qv_lo = vwmul_vv_i16m2(q4_lo, q8_lo, 32)      // 1 inst → 32×i16
  vs_lo = vredsum_vs_i16m2_i16m1(qv_lo, vzero, 32) // 1 inst → 1×i16
```

```
    sum += vmv_x_s(vs_lo) * scales[j]              // 1 inst (scalar mul)
    Total: 4 inst, i16 精度

  After (with vdot_scale):
    vdot_scale(acc_i32, q4_x, q8_lo, scales[j], 8); // 1 inst → 8×i32
    Total: 1 inst, i32 精度
```

指令减少: Per j iteration: 4×2 (两半) + 2 (load) + 2 (nibble extract) → 1×2 + 2 (load) + 2 (nibble extract) = 6 inst. **BB**内减少: (14 - 6) / 14 = **57.1%**

实现难度: 高。需要新的执行单元（pairwise dot + widening + multiply-by-scalar），且输出 EEW=32 与输入 EEW=8 的比例关系需要新的 LMUL 约束处理。

[P2] 矩阵乘累加 **(MMA) —** 长期架构方向

指令定义:

```
格式: vmmla_s32 acc, vs2_row, vs1_col, vm
语义: acc[i][j] += zext(vs2_row.B[i]) * sext(vs1_col.B[j])  for i,j ∈ [0,3]
输入: acc (i32 4×4 matrix accumulator), vs2 (i8 row, 4 elements), vs1 (i8 col, 4 elements)
输出: acc (updated 4×4 i32 matrix)
约束: 需要 4×LMUL 寄存器组作为矩阵累加器
```

对应: ARM `svmmla_s32` (SVE2+MATMUL), Power `xvf32gerpp` (POWER10 MMA)

收益: 将 64-element subblock 拆分为 4×4 块，每块 1 次矩阵 MAC。 - Before: 56 inst/super-block - After: ~8 inst/super-block (4 subblocks × 2 MMA each) - **BB**内减少: (56 - 8) / 56 = **85.7% (**理论值**)**

实现难度: 极高。需要专用矩阵累加器寄存器（类似 POWER10 ACC），新的矩阵执行单元，以及复杂的寄存器分配策略。这是多代硬件演进的长期目标。

[P2] 专用累加器寄存器

对应: Power MMA ACC0-ACC7 (512-bit), LoongArch 向量域累积模式

需求: 提供不参与通用向量寄存器分配的专用累加器，消除 vredsum 开销。

收益: 消除每次迭代的 reduce-to-scalar 操作，允许跨迭代向量域累积。

实现难度: 高。需要修改 ISA 寄存器文件设计，增加读写端口和转储/恢复机制。

附录

A. 融合点积指令跨平台对比

平台	指令	**u8×i8**	**Pairwise**	输出精度	**Accumulate**	累积域
x86 AVX2	PMADDUBSW	Yes	Yes (pairs)	i16	No	向量
x86 AVX2	PMADDWD	—	Yes (pairs)	**i32**	No	向量
ARM NEON	vdotq_s32	Yes	Yes (groups of 4)	**i32**	Yes	向量
ARM SVE2	svmmla_s32	Yes	4×4 matrix	**i32**	Yes	向量
Power	vec_msum	Yes	Yes (groups of 4)	**i32**	Yes	向量
Power MMA	xvf32gerpp	Yes	4×4 matrix	**i32**	Yes	专用 ACC
LoongArch	madd_h_b	Yes	No	i16	Yes	向量
S390X	VMSL	Yes	No	i16	Yes	向量
S390X	VMSLG	Yes	No	**i64**	Yes	向量

平台	指令	**u8×i8**	**Pairwise**	输出精度	**Accumulate**	累积域
WASM	dot_i16x8	i16 level	Yes (pairs)	**i32**	Yes	向量
RVV	**vwmul+vredsum**	**Yes**	**No**	**i16**	**No**	标量

B. 数据重排指令对比

操作	**ARM**	**x86**	**Power**	**LoongArch**	**RVV**	差距
Nibble extract (AND mask)	vandq_u8	and_si256	vand	xvand.v	vand_vx	无
Nibble extract (shift)	vshrq_n_u8	srli_epi16	vsrab	xvsrani.b	vsrl_vx	无
Byte shuffle	vqtbl1q	pshufb	?	xvshuf.b	vrgather	RVV 更灵活

结论: Nibble 提取各平台均无显著优势。RVV 的 `vrgather` 比固定模式的 `pshufb` 更灵活。

C. 加载/存储指令对比

操作	**ARM**	**x86**	**Power**	**RVV**	差距
Vector load	vld1q	loadu_si256	lxvd2x	vle8	RVV 更灵活（可变长度）
Load + broadcast	vld1q_dup	(via shuffle)	vlrep	vle8_v + vmv	

操作	**ARM**	**x86**	**Power**	**RVV**	差距
					ARM/Power 更直接
Stride load	vld1q (strided)	vmovdqa (aligned)	lxvdsx	vlse8	RVV 原生支持

结论: 加载/存储 RVV 全面领先，原生支持可变长度和步长加载。

结论

核心发现

1. **RVV** 最大的问题不是缺少指令，而是代码未充分利用现有指令。`vwmacc_vx`（规范已有）可以实现向量域 i32 累积，消除 reduce-to-scalar 瓶颈，减少 35.7% 主循环指令。这是零硬件成本的最大收益。
2. 跨平台共识: 6/6 个非 RVV 平台都有融合点积指令。i8×i8→直接精度（i32）的融合乘加是业界标准，RVV 是唯一需要分离 vwmul + vredsum 的平台。
3. 精度差距是真实的安全问题: i16 中间精度（最大值 ±6144，范围 ±32767，安全裕度仅 5.3×）在极端输入下可能溢出。多平台的 i32 直接产出消除了此风险。

推荐实施路径

阶段	行动	收益	成本
立即	循环重构使用 `vwmacc_vx`	-35.7% 指令 + i32 精度	纯代码修改
短期	引入 `vwmaccbb_vv` 扩展	额外 -14.3%（组合 -50%）	新 ISA 扩展
		组合 -57.1%	

阶段	行动	收益	成本
中期	引入 `vdot_scale`（i8→i32 融合）		复杂 ISA 扩展
长期	MMA + 专用累加器	理论 -85.7%	全新硬件单元

审查日志

Review Round 1 (self-review, verified against RVV intrinsic spec)

RVV Intrinsic Verification: - `vwmacc_vx_i32m4(vint32m4_t vd, int16_t rs1, vint16m2_t vs2, size_t vl)` — confirmed in `intrinsic_funcs.adoc:34796` - `vwmul_vv_i16m2(vint8m1_t vs2, vint8m1_t vs1, size_t vl)` — confirmed in `intrinsic_funcs.adoc:32955` - `vredsum` LMUL reduction pattern — confirmed in overloaded test files

轮次	发现问题数	已修复	剩余
R1	3	1	0

Issue 1 [MAJOR] — Fixed

- **Location**: Section "3. LoongArch LASX/LSX", LASX code example and instruction count table
- **Problem**: `madd_h_b` described as 4-operand instruction with scale parameter (`madd_h_b(acc, q4, q8, scale)`). Real LASX `XMADD_H_B` is 3-operand (`xd.H[i] += xs1.B[i] * xs2.B[i]`), no scale. This inflated the benefit from ~21.4% to 35.7%.
- **Expected**: Correct 3-operand form, separate scale broadcast and multiply instructions
- **Evidence**: LoongArch ISA specification — `XMADD_H_B xr, xrj, xrk`

- **Fix**: Corrected code example to show 3-operand `madd_h_b`, added separate `xvmul_w` for scale, updated instruction count from 9 to ~11 and benefit from 35.7% to 21.4%.

Issue 2 [MINOR] — Accepted

- **Location**: Section "基准RVV实现分析", per-iteration count of 14
- **Problem**: The count of 14 bundles compound operations (vmv_x_s + scalar mul counted as 1, pointer updates counted as 1 each). Actual distinct C intrinsic calls: 11 vector intrinsics + 2 pointer updates + scalar operations.
- **Note**: This counting convention is consistent with the x86 AVX2 analysis (which also bundles compound operations). The comparison (14 vs 14) is internally consistent and fair.

Issue 3 [MINOR] — Accepted

- **Location**: Section "[P0] 循环重构使用 vwmacc_vx", proposed code
- **Problem**: `scales[2*j]` (uint8_t) is passed as `int16_t rs1` to `vwmacc_vx_i32m4`. The scalar is sign-extended from the X register.
- **Note**: Q4_K scales are unsigned, range 0-15 (6-bit packed). Zero-extension to int16 gives the same value as sign-extension for this range. No correctness issue for this operator.

Review Round 2 (self-review, focused verification)

All claims re-verified: - Summary table numbers match detailed analysis sections: confirmed - Benefit arithmetic (35.7% = (14-9)/14, 14.3% = 2/14): confirmed - All RVV intrinsic names and operand types: confirmed against spec - All platform instruction names: correct per public ISA documentation - AVX2 per-iteration count (14): re-counted as 14 (x86 sub-analysis had minor counting error at 13, corrected in this report)

轮次	发现问题数	已修复	剩余
R2	0	—	0

最终审查结论：审查通过。零 CRITICAL、零 MAJOR 问题。2 个 MINOR 问题已确认为可接受的计数约定和类型安全限制。

参考文档

- x86 AVX2 分析: `applications/llama.cpp/temp/x86-avx2-analysis.md`
- ARM NEON/SVE 分析: `applications/llama.cpp/temp/arm-neon-sve-analysis.md`
- LoongArch LASX/LSX 分析: `applications/llama.cpp/temp/loongarch-lasx-analysis.md`
- Power VSX/MMA 分析: `applications/llama.cpp/temp/power-vsx-analysis.md`
- S390X Z-Vector 分析: `applications/llama.cpp/temp/s390x-zvector-analysis.md`
- WASM SIMD 分析: `applications/llama.cpp/temp/wasm-simd-analysis.md`
- RVV 基准实现: `rvv-patches/vec_dot-q4_K_q8_K/rvv_vec_dot_q4_K_q8_K.inl`
- llama.cpp 源码: `vendor/llama.cpp/ggml/src/ggml-cpu/arch/riscv/quants.c`